

Projektverlauf: Vom Entwurf zur generalisierungsfähigen Navigation - der Weg der Arbeit

Basis: Analyse der vollständigen Git-Historie (139 Commits, 09.03.–09.07.2026) über die Branches Milestone-1,2, Milestone-3-Agent-Grundsystem, milestone-4-hindernisse-und-interaktion, milestone-5-reward-system-training, milestone-7, Messe2 und ModelTrainingComparsion sowie der Trainings- und Analyseartefakte des Repositories. Die Gliederung folgt der inhaltlichen Logik der Arbeit; Datums- und Branch-Angaben verorten jeden Schritt in der Historie. Stand: 10.07.2026.

1. Erster Entwurf und Zielbild (März, Branch main)

Am Anfang steht die Projektskizze vom 9. März: Ein Agent soll lernen, in Labyrinthen mit Gefahrenelementen zu einem Ziel zu navigieren - als testbare Abstraktion realer Navigationsszenarien. Daraus leiten sich die drei Leitfragen der Arbeit ab: der Mehrwert temporalen Gedächtnisses gegenüber einer gedächtnislosen Baseline (F1), der direkte Vergleich der Gedächtnisarchitekturen LSTM und Transformer (F2) und die Generalisierung auf unbekannte Karten (F3). Diese Fragen bestimmen den gesamten weiteren Weg: Erst wird eine kontrollierbare Umgebung gebaut, dann werden die drei Architekturen einzeln lauffähig gemacht und optimiert, schließlich unter kontrollierten Bedingungen gegenübergestellt und auf Generalisierung geprüft. In den ersten Märzwochen entstehen parallel Roadmap, Issue-Struktur und Dokumentationsgerüst.

2. Die Umgebung: Map-System und prozedurale Generierung (Milestones 1, 2 und 6)

Die Umgebung ist die erste Baustelle, denn ohne kontrollierbare, variierende Karten ist weder Training noch Generalisierungsmessung möglich. Im Branch Milestone-1,2 (März) entsteht der MapGenerator: Karten werden beim Szenenstart aus Layout-Definitionen erzeugt, mehrere Layouts werden unterstützt, Spawn- und Zielelemente platziert, ein Laufzeit-Reset ermöglicht Episodenwechsel. Nach einem dokumentierten Generierungs-Rework Mitte April (Milestone 4) wird die Erzeugung in Milestone 6 (April/Mai, im Branch milestone-7 weitergeführt) auf prozedurale Beine gestellt: Ein Raum-Korridor-Graph erzeugt Topologien, ein semantischer Pathfinder validiert die Lösbarkeit jeder Karte, ein Schwierigkeitsgrad-Enum systematisiert die Kartenklassen von Trivial bis Hard, und ein Objekt-Pool hält die Laufzeitkosten der Kartenwechsel gering. Entscheidend für die spätere Generalisierungsmessung: Karten entstehen seed-basiert und damit reproduzierbar - Trainings- und Held-out-Kartensätze lassen sich sauber trennen. Mit dem CurriculumTracker kommt zugleich die phasenweise Schwierigkeitssteigerung ins System; Konstanten wie der globale Seed 42 und phasenspezifische Episodenlängen werden fixiert.

3. Der Agent: Grundsystem, Gefahren, Sensorik und Belohnung (Milestones 3, 4 und 5)

Auf die Umgebung folgt der Agent. Der Branch Milestone-3-Agent-Grundsystem (Anfang April) bringt die zentrale Klasse LabyrinthAgent mit ML-Agents-Anbindung, diskretem Aktionsraum (Bewegen, Drehen, Springen) und Rigidbody-Physik. Der Branch milestone-4-hindernisse-und-interaktion macht die Navigation riskant: Lava, Löcher und Brücken werden als Zelltypen mit Tags und Todeslogik eingeführt, der RayPerceptionSensor nimmt die Umgebung über diese Tags wahr, eine Sensor-Testszene sichert die Erkennung ab. Der parallel integrierte Branch milestone-5-reward-system-training ergänzt die Belohnungsfunktion (Zielprämie, Todesstrafen, Zeitstrafe, Distanz-Shaping) und die erste PPO-Trainingskonfiguration. Damit ist Mitte April die vollständige Trainingsschleife geschlossen - und alle drei später verglichenen Architekturen

werden auf exakt diesem gemeinsamen Unterbau aufsetzen.

4. Architektur I - MLP-Baseline: Implementierung und Optimierung (Milestone 5, Fortführung in milestone-7)

Das Multilayer-Perzeptron ist als native ML-Agents-Standardarchitektur die am schnellsten einsatzfähige der drei Varianten und übernimmt zwei Rollen: technischer Nachweis der Pipeline und unterer Vergleichsanker für F1. Direkt nach Milestone 5 laufen drei Baseline-Trainings mit identischer Konfiguration: `test_run` validiert die Pipeline (Unity-Anbindung, Checkpoints, TensorBoard), `mlp_baseline_v1` wird nach rund 600.000 Steps abgebrochen, weil die Umgebung parallel weiterentwickelt wurde und der Lauf damit nicht mehr auf eingefrorenem Stand basierte - die erste dokumentierte methodische Lektion des Projekts. Der Referenzlauf `mlp_baseline_v2` erreicht über zwei Millionen Steps einen mittleren Episoden-Reward von etwa +0,81 und läuft erwartungskonform in ein Plateau: Eine reaktive, gedächtnislose Policy kann mehrdeutige Situationen der teilbeobachtbaren Umgebung nicht auflösen. Im Mai folgt im Branch `milestone-7` ein MLP-Langzeittraining über rund 20,8 Millionen Steps als belastbare Referenz. Optimierungsbedarf im engeren Sinn besteht beim MLP kaum - genau diese Einfachheit macht es als Baseline wertvoll.

5. Architektur II - LSTM: native Integration und Iterationen (milestone-7)

Das LSTM nutzt die im Framework vorgesehene Memory-Konfiguration (`memory_type: lstm`) und ist entsprechend schnell lauffähig. Die Optimierung erfolgt über die Iterationsserie `lstm_v1` bis `lstm_v7b` sowie eine Curiosity-Variante. Zentrale Erkenntnisse dieser Serie, die bis in den finalen Vergleich Bestand haben: die Beobachtungsnormalisierung (`normalize: true`) stabilisiert den Encoder im Curriculum-Betrieb, `sequence_length 16` und `memory_size 256` erweisen sich als tragfähige Gedächtnisdimensionierung, und ein längerer `time_horizon (512)` passt zu den langen Episoden der Aufgabe. Der Integrationsaufwand bleibt insgesamt gering - eine Asymmetrie zum Transformer, die später in der Methodik explizit reflektiert wird.

6. Architektur III - Transformer: Nachrüstung und der lange Weg V1-V24 (milestone-7, Messe2)

Der Transformer ist die aufwendigste Architektur, denn ML-Agents bringt keine Transformer-Memory-Policy mit. Statt eines Framework-Forks wird die Policy über ein Patch-Skript reproduzierbar in die Python-Umgebung nachgerüstet (`patch_mlagents.py` kopiert `transformer_memory.py` und erweitert `settings.py/networks.py`; `start_training.py` wendet den Patch automatisch an). Es folgt die iterationsreichste Phase des Projekts, dokumentiert als Serie V1 bis V24 (Mai): Bis V13 wird die Integration lauffähig - Framework-Workarounds, die ONNX-Export-Kette für die Unity-Inferenz (Batch-Size-1-Export, LayerNorm-Korrekturen) und die Konsistenz zwischen Training und Inferenz (Rolling-Memory-Buffer) sind die Hauptbaustellen. V14 absolviert erstmals einen vollständigen Curriculum-Durchlauf über gut 31 Millionen Steps und scheitert diagnostizierbar: insbesondere am Übergang in kombinierte Gefahrenszenarien, begleitet von Reward- und Curriculum-Pathologien wie dem Aussitzen von Phasen. V15 bis V24 arbeiten diese Diagnosen ab - Hyperparameter-Korrekturen (Lernrate, Entropie-Koeffizient), Reward-Anpassungen und Curriculum-Umbauten. Als Nebenprodukt entsteht Ende Mai aus dem V24-Stand der Messe-Demonstrator (Branch `Messe2`): eine Wettkampfszene Mensch gegen Agent mit eigenem Kompatibilitäts- und Competition-Modus. Danach ruht das Repository im Juni - die Pause trennt die

explorative Einzeloptimierung vom kontrollierten Vergleich.

7. Gegenüberstellung: ein gemeinsamer Lauf, identische Bedingungen (ModelTrainingComparsion)

Für die eigentliche Vergleichsphase (Juli) entsteht der Branch ModelTrainingComparsion mit einer Standalone-Trainingspipeline: Alle drei Architekturen trainieren als drei Behaviors gleichzeitig im selben Environment - gleiche Karten, gleiche Episoden, gleiches Curriculum, gleicher Seed. Headless-Builds, Watchdog-Neustarts, parallele Environment-Prozesse, stündliche Analyse-Reports und TensorBoard-Monitoring machen mehrtägige Läufe praktikabel. Die Unity-Seite wird vollständig kontrolliert: identisches Agentenskript, identische Observations, identische Belohnungsfunktion für alle drei. Die ersten Vergleichsläufe (model_comparison_v1 bis v4) starten bewusst mit weitgehend identischen Trainer-Konfigurationen - und decken dabei zunächst eine Serie von Infrastrukturproblemen auf, die diagnostiziert und behoben werden: eine Absturzscheife durch Speichererschöpfung beim simultanen Checkpoint-Export, der Verlust des Curriculum-Fortschritts bei Prozessneustarts (behoben durch datebasierte Persistenz je Worker) sowie Bereinigungen an Curriculum-Konfiguration und Episoden-Timeouts.

8. Der methodische Pivot: Configs vergleichbar machen - und dann das Maximum herausholen

In der Auswertung der identisch konfigurierten Läufe zeigt sich ein methodisches Dilemma: Erzwangene 100%-Identität der Trainer-Parameter ist Scheinfairness, denn Werte, die MLP und LSTM gut dienen (etwa Lernrate $3e-4$), lähmen den Transformer nachweislich - seine Einzeloptimierung hatte andere Arbeitspunkte ergeben. Die Konsequenz ist der dokumentierte Wechsel auf das Prinzip Best vs. Best: Konstitutiv verschieden ist einzig memory_type (ohne ihn gäbe es keinen Vergleichsgegenstand); daneben erhält jede Architektur ihre jeweils beste bekannte Konfiguration aus der eigenen Optimierungshistorie, jede Abweichung einzeln begründet und tabelliert. Der erste finale Lauf unter diesem Design (rund vier Millionen Steps, anschließend archiviert) liefert den entscheidenden Befund: MLP und LSTM lernen die Aufgabe, der Transformer bleibt trotz seiner erprobten Konfiguration nahezu erfolglos - weil seine Konfiguration für eine Umgebung mit Zielrichtungs-Observation entstanden war, die im Vergleichsaufbau fehlte. Der zweite finale Lauf korrigiert das: Der Zielrichtungsvektor kehrt für alle drei Architekturen identisch in den Beobachtungsraum zurück, ein Loch-Vermeidungs-Penalty ergänzt die Belohnung, und das Curriculum wächst auf acht Phasen mit isolierten Gefahren-Lernstufen, Erfolgsraten-Gates und Wiederholungsschleife. Das Ergebnis bestätigt die Diagnose: Alle Gates werden regulär bestanden, alle drei Architekturen lösen die schwersten Kartenklassen mit block-über-block steigenden Erfolgsraten - das LSTM vorn, der zuvor gescheiterte Transformer nun klar lernfähig.

9. Zu guter Letzt: Generalisierung

Die Generalisierungsfrage (F3) ist der Schlussstein des Wegs - und war von Anfang an in die Systementscheidungen eingebaut: Die egozentrische, lokale Sensorik ohne globale Karteninformation verhindert Kartenauswendiglernen strukturell; zufällige Spawn- und Zielpositionen machen jede Episode auch auf bekannten Layouts zu einer neuen Navigationsaufgabe; die seed-basierte prozedurale Generierung erlaubt es, Held-out-Kartensätze zu erzeugen, die im Training garantiert nie gesehen wurden. Auf dieser Grundlage sieht das Evaluationsprotokoll die Messung eines Overfitting-Index vor: Erfolgsraten auf Trainingskarten

gegen frisch generierte Karten derselben Verteilung, je Architektur. Ergänzend entstand ein autarkes Training-Kit, das den kompletten Trainingsaufbau reproduzierbar auf Fremdhardware ausführbar macht - Reproduzierbarkeit als praktische Facette der Generalisierung. Die Ergebnisse dieser Messungen bilden den Abschluss der Evaluation (Kapitel 8) und beantworten die dritte Forschungsfrage.

10. Der Weg im Rückblick

In der Zusammenschau folgt die Arbeit einer klaren Logik: Entwurf und Forschungsfragen (März) - kontrollierbare, prozedural generierte Umgebung (M1/2, M6) - gemeinsamer Agenten-Unterbau mit Gefahren, Sensorik und Belohnung (M3-M5) - Einzelimplementierung und -optimierung der drei Architekturen mit stark asymmetrischem Aufwand (MLP: nativ und schnell; LSTM: native Memory mit kurzer Iterationsserie; Transformer: Nachrüstung mit langer Serie V1-V24) - kontrollierte Gegenüberstellung in einem gemeinsamen Lauf - der methodische Pivot von erzwungener Parameteridentität zu begründetem Best vs. Best - und schließlich die Generalisierungsprüfung. Durchgängiges Muster ist der Umgang mit Abweichungen: Sie wurden nicht kaschiert, sondern dokumentiert, diagnostiziert und in Verbesserungen überführt - vom Map-Rework über die V14-Diagnose bis zur Beobachtungsraum-Korrektur zwischen den beiden finalen Läufen.